

CS5351 2025/26 学期A

现代代码审查

黄志安教授

邮箱: huang.za@cityu-dg.edu.cn

什么是代码审查

- ◆ 由非代码作者的开发者对源代码进行人工审查。
 - ◆ 是软件工程领域成熟的实践方法
 - ◆ 旨在通过知识转移来**维持和提升源代码质量**，并通过知识转移来**维持开发社区**
- 他人采用的设计与实现方案[12]

课程大纲

- ◆ 什么是代码审查？
- ◆ 为何需要代码审查？
- ◆ 代码检查
- ◆ 现代代码审查
- ◆ 背景阅读
 - https://en.wikipedia.org/wiki/Fagan_inspection
 - <https://ieeexplore.ieee.org/document/6606617/>
 - 连接校园网络时可下载
- ◆ 尝试学生团队开发的代码审查工具（如MCR-2）

什么是代码审查

- ◆ 由非代码作者的开发人员对源代码进行人工检查。

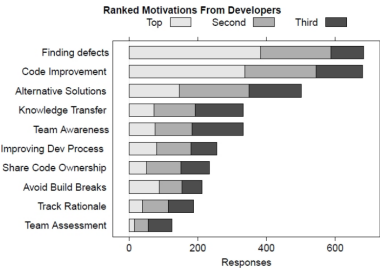


Fig. 3. Developers' motivations for code review. [1]

除发现缺陷外，代码审查还具有多重目的：代码优化、替代方案探索、知识传承、团队建设等。

不同层级的正式性

代码演示

- 非正式会议，程序员带领审查团队审阅其代码，审查者尝试找出缺陷
- 若程序员与评审者非同一人，效果更佳
- 有时可与结对编程结合

代码检查

- 正式会议
- 能有效发现代码中的错误。
- 课后阅读：https://en.wikipedia.org/wiki/Fagan_inspection

现代代码审查

```
bool isWin(char game[3][3]){
    bool win = false;
    if (game[0][0] == game[0][1] && game[0][1] == game[0][2] && (game[0][0] == 'X' || game[0][0] == 'O')) win = true;
    if (game[1][0] == game[1][1] && game[1][1] == game[1][2] && (game[1][0] == 'X' || game[1][0] == 'O')) win = true;
    if (game[2][0] == game[2][1] && game[2][1] == game[2][2] && (game[2][0] == 'X' || game[2][0] == 'O')) win = true;
    // 纵列
    if (game[0][0] == game[1][0] && game[1][0] == game[2][0] && (game[0][0] == 'X' || game[0][0] == 'O')) 横走 = true;
    if (game[0][1] == game[1][1] && game[1][1] == game[2][1] && (game[0][1] == 'X' || game[0][1] == 'O')) win = true;
    if (game[0][2] == game[1][2] && game[1][2] == game[2][2] && (game[0][2] == 'X' || game[0][2] == 'O')) win = true;
    // 对角线
    if (game[0][0] == game[1][1] && game[1][1] == game[2][2] && (game[0][0] == 'X' || game[0][0] == 'O')) 横走 = true;
    if (game[0][2] == game[1][1] && game[1][1] == game[2][0] && (game[0][2] == 'X' || game[0][2] == 'O')) win = true; return win;
}

int main(){
    int i, j;
    char game[3][3] = {' '}; // 开字棋char player1 = 'X';
    char player2 = 'O';
    bool turn = true; // 玩家:回合时为false, 玩家:回合时为true, 玩家:先手. cout << "X = 玩家1" << endl << "O = 玩家2" << endl;
    for (int n=0; n<9; n++){
        turn = !turn; // 使用非运算符将true转为false或false转为true, if (turn == false)
            cout << "玩家1:" << endl;
        else
            cout << "玩家2:" << endl;
        cout << "选择并记录那个单元格? (1[1..3], 2[1..3] 1: *j cin >> i >> j);
        if (turn == false) game[i][j] = 'X';
        else
            game[i][j] = 'O';
        if (isWin(game)){
            cout << "获胜!" << endl;
            break; // 提前终止问题
        }
    }
    if (i==3) // 所有1~9的单元格已输入但未获胜 cout << "平局!" << endl;

    // 将游戏状态输出至控制台
    cout << game[0][0] << " " << game[0][1] << " " << game[0][2] << endl;
    cout << game[1][0] << " " << game[1][1] << " " << game[1][2] << endl;
    cout << game[2][0] << " " << game[2][1] << " " << game[2][2] << endl; return 0;
}
```

代码解析:

isWin() 的目标是实现什么功能, 因此.....

Main() 在这几行代码中执行此操作, 在那些行中执行那项操作

代码解析

来自一篇旧论文的结果[21]

- 类似于测试在发现缺陷方面的有效性
- 但代码审查需要更多人力投入, 因此效率较低

近年来, 开发者开始关注技术债务 (非缺陷问题), 而测试对此类问题收效甚微

- 例如: 可读性、可复用性、可维护性等

远程协作因"居家办公"需求兴起, 使视障开发者能远程参与代码审查。后续幻灯片将展示若干应对策略。

例如: 代码走查[22]

Table 3 Use of audio cues to convey awareness indicators in Follow mode (unless specified otherwise in the row)

Awareness Information	Non-Speech Indicator	Non-Speech Indicator Frequency	Speech Indicator	Speech Indicator Frequency	Built-in Visual Indicator
Viewport scrolls	Click wheel event	Every scroll	None	None	Screen scrolls
Scroll direction	Falling or rising tone depending on direction	When scrolling stops	None	None	Can be inferred from the scrolling viewport
Current Viewport	None	None	"Lines X to Y on screen"	When scrolling stops	Visible on screen
Cursor moves by single line to line N	Keyboard click	Every cursor move	"Line N"	1.5 seconds after cursor moves end	Cursor moves on screen
Cursor moves multiple lines to line N	Falling or rising tone depending on move direction	Every event	"Line N"	1.5 seconds after cursor moves end	Cursor moves on screen
Cursor moves by multiple lines to line N	Falling or rising tone depending on move direction	Every event	"Line N"	Every event	Cursor moves on screen
Selection	Depends on selection (they heard mouse)	Every event	"Selection on line N"	1.5 seconds after selection is made	Selection visible on screen
Edit on follower's line	Keyboard type	For every character typed	None	None	Cursor moves on screen, edits visible on screen
Edit on follower's line (Follow mode off)	Keyboard type	For every character typed	"collaborator is editing the same line as you"	As long as edits continue on the same line	Cursor moves on screen, edits visible on screen
Edit within 3 lines of follower (Follow mode off)	Proximity sound	For every character typed	"collaborator is editing nearby"	As long as edits continue on the same line	Cursor moves on screen, edits visible on screen
Follow status	Pull and push sound	When follower starts and stops following leader	"You are now following -collaborator"	Every event	None



代码审查[6]

- ◆ 是一种高度结构化的正式审查流程
 - 企业中常称为*静态测试*
 - 有效发现缺陷并识别进一步的流程改进机会
 - 日益失宠
- ◆ 核心要点
 - **会前**：作者向评审者讲解，评审者从测试角度阅读/质疑代码。
 - **会议中**：由代码作者带领审阅者逐行走代码，对照检查清单核对。解答疑问并消除歧义。每位审阅者在会前被分配预定义角色（如负责检查特定重点领域）。
 - 评审员在脑中“执行”代码
 - **会后**：作者跟进并修正代码。会议主持人核查所需整改措施是否落实到位。
- ◆ 特征
 - 正式、同步、代码检查为主、工具辅助为辅、权重较高⁹

代码审查

- ◆ 检查要点
 - 算法效率
 - 代码逻辑与正确性
 - 例如：控制流与数据流、输入输出格式与顺序、参数与返回值类型及顺序等
 - 注释与代码一致性
- ◆ 制定自己的检查清单

MCR：作者私人材料

井字棋（過三關）是一款双人游戏。游戏中存在一个3x3的网格。初始时网格为空。每个单元格可放置字符'x'、字符'o'或保持空置。当出现横向、纵向或对角线出现相同符号全数填满的情况，则该方获胜。例如图示中底部横排全被"x"符号占据，故"x"方获胜。

	x	o	x	o	x	o	x	o	x	o	x
			x		x		x	x	x	x	x

该程序允许两名玩家（玩家1和玩家2）轮流输入所选格子的坐标。每次接收输入后，程序需判断是否存在胜者，若有胜者则输出消息“胜！”。若填满全部9个格子后仍无胜者，则输出“平局！”。在显示"胜！"或"平！"后，立即显示 (1) s(1) 整个井字棋棋盘并结束游戏。

MCR:

作者需提交给审稿人的材料

控制台界面：

```
X = 玩家1 O = 玩家
2
玩家1: 标记哪个单元格? i:[0..2], j:[0..2]: 0 2
玩家2: 标记哪个单元格? i:[0..2], j:[0..2]: 0 0
玩家1: 标记哪个单元格? i:[0..2], j:[0..2]: 2 0
玩家2: 标记哪个单元格? i:[0..2], j:[0..2]: 1 1
玩家1: 标记哪个单元格? i:[0..2], j:[0..2]: 2 2
玩家2: 标记哪个单元格? i:[0..2], j:[0..2]: 1 2
玩家1: 标记哪个单元格? i:[0..2], j:[0..2]: 2 1 获胜!
O   O
  O O
X X X
```

输入内容以下划线标注



任何意见/建议（5分钟）

```
def is_win(game):
    win = False
    # Check rows
    if game[0][0] == game[0][1] == game[0][2] and (game[0][0] == 'X' or game[0][0] == 'O'):
        win = True
    if game[1][0] == game[1][1] == game[1][2] and (game[1][0] == 'X' or game[1][0] == 'O'):
        win = True
    if game[2][0] == game[2][1] == game[2][2] and (game[2][0] == 'X' or game[2][0] == 'O'):
        win = True
    # Check columns
    if game[0][0] == game[1][0] == game[2][0] and (game[0][0] == 'X' or game[0][0] == 'O'):
        win = True
    if game[0][1] == game[1][1] == game[2][1] and (game[0][1] == 'X' or game[0][1] == 'O'):
        win = True
    if game[0][2] == game[1][2] == game[2][2] and (game[0][2] == 'X' or game[0][2] == 'O'):
        win = True
    # Check diagonals
    if game[0][0] == game[1][1] == game[2][2] and (game[0][0] == 'X' or game[0][0] == 'O'):
        win = True
    if game[0][2] == game[1][1] == game[2][0] and (game[0][2] == 'X' or game[0][2] == 'O'):
        win = True
    return win

def main():
    game = [[' ' for _ in range(3)] for _ in range(3)] # Tic-tac-toe board
    player1 = 'X'
    player2 = 'O'
    turn = False # False for player 1's turn, True for player 2's turn. Player 1 first.
    print("X = Player 1")
    print("O = Player 2")
    for n in range(9):
        turn = not turn # Switch turns
        if not turn:
            print("Player 1: ", end="")
        else:
            print("Player 2: ", end="")
        print("Which cell to mark? i[1..3], j[1..3]: ")
        i, j = map(int, input().split())
        i -= 1
        j -= 1
        if not turn:
            game[i][j] = 'X'
        else:
            game[i][j] = 'O'
        if is_win(game):
            print("Win!")
            break # Terminate the game
        if n == 8: # All cells have been filled
            print("Tie!")
        # Show the game board
        for row in game:
            print(" ".join(row))

if __name__ == '__main__':
    main()
```

现代代码审查（MCR）

- ◆ MCR是一种轻量级且基于工具的代码变更审查（而非整段代码！），
 - 这是许多软件开发项目的标准做法
- ◆ MCR的特征
 - 非正式；工具驱动的流程；异步进行；专注于代码变更审查；比代码检查更轻量级
- ◆ 核心理念
 - 作者A对代码块C进行补丁P修改以解决问题，并通过电子渠道（如邮件、WhatsApp、微信或GitHub工具）将补丁P发送给评审组R
 - 每位审阅者R评估P：认可P或拒绝P
 - 作者A提交P；提交后其他R可进行审阅。
- ◆ 作者与审稿人交流观点、发现错误并探讨替代方案

为提交的代码变更提供更优的结构设计方案。

MCR流程

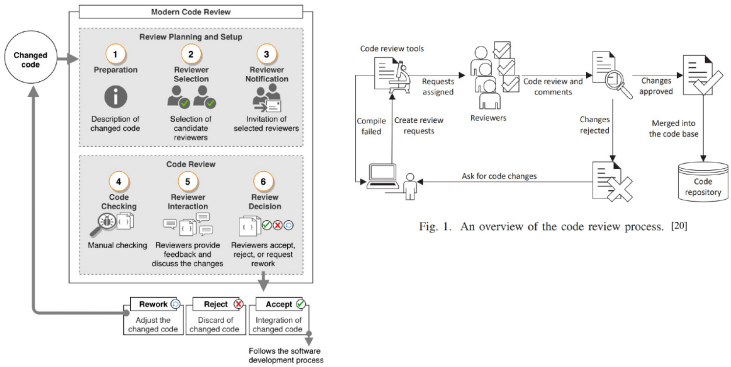
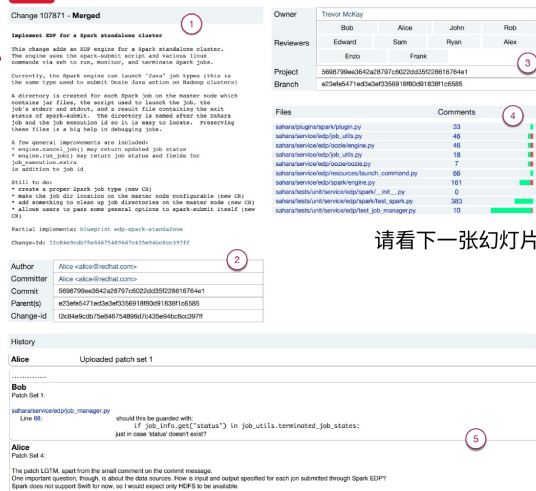


Fig. 1. An overview of the tasks of modern code review. [19]

Fig. 1. An overview of the code review process. [20]

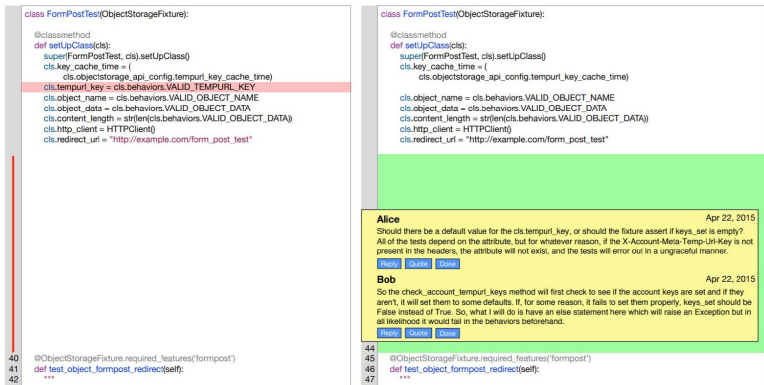
示例 [12]



请看下一张幻灯片

示例[12]

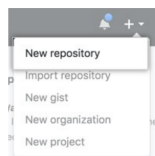
在变更中展开文件



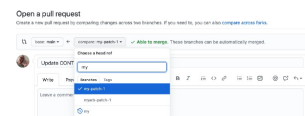
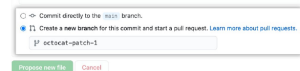
示例 [17]



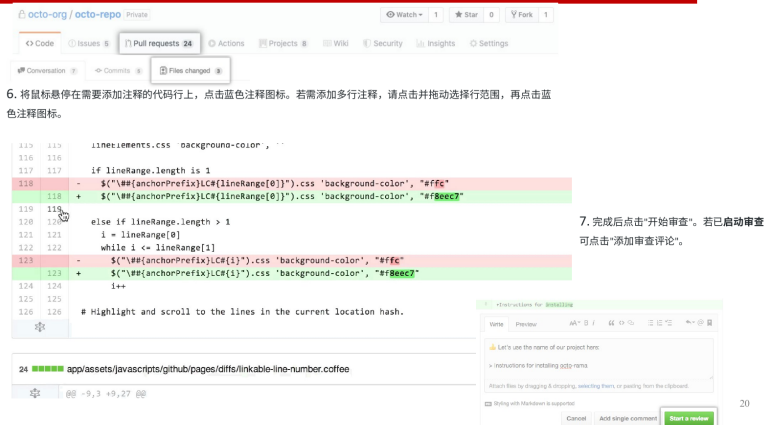
1. 在任意页面的右上角，使用下拉菜单，选择新建存储库。
2. 为您的存储库输入简短易记的名称。例如“hello-world”。
3. 对文件进行所需修改。



4. 决定是否将您的提交添加到当前分支或新建分支。



示例 [17]



- ◆ 非正式
 - 由临时组建的评审小组进行，成员无预设角色，评审流程不遵循正式程序
- ◆ 工具驱动的后勤支持
 - 电子邮件、WhatsApp、微信广受欢迎。
 - 开源工具：GitHub、Gerrit（用于Git）、ReviewBoard、Phabricator
- ◆ 异步协作
 - 作者与审阅者无需同时处理同一任务（但不可过度延迟）
- ◆ 审阅聚焦于补丁（代码变更，增量部分）

Comments in each Category

Category	Percentage of Comments
Code Improvement	~29%
Understanding	~22%
Social Communication	~16%
Defects	~14%
External Impact	~5%
Testing	~5%
Review Tool	~3%
Knowledge Transfer	~2%
Misc	~7%

四大核心领域

Fig. 4. Proportion of comments by card sort category.

应对困惑的策略 [14]

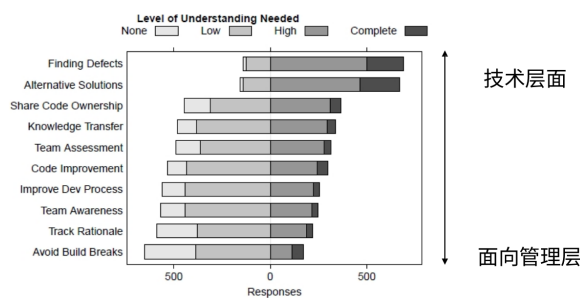
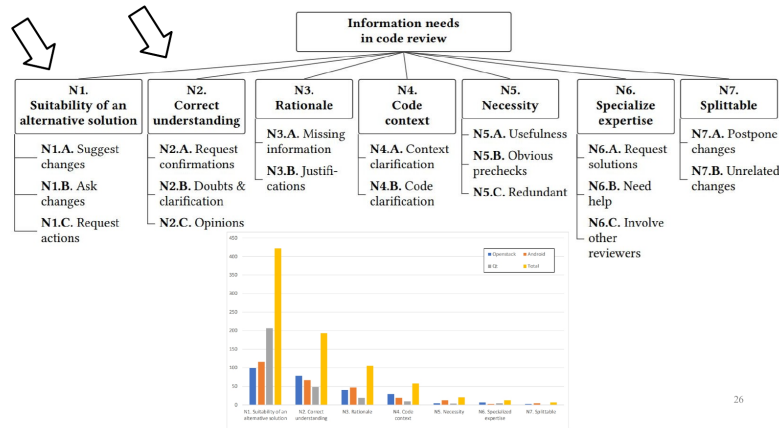


Fig. 5. Developers responses in surveys of the amount of code understanding for code review outcomes.

THE REASONS, IMPACTS AND COPING STRATEGIES DEVELOPERS USE TO DEAL WITH CONFUSSION; IN THE PARENTHESIS ARE THE NUMBERS OF CARDS.				
	Review process 18 topics (120)	Artifact 15 topics (300)	Developer 15 topics (124)	Link 9 topics (177)
	<u>Organization of work (17)</u> Issue tracker, version control (7) Unnecessary change (6) Not enough time (3) Dependency between changes (3) Code ownership (2) Community norms (2)	Missing rationale (66) Discussion of the solution: non-functional (49) Unsure about system behavior (37) Lack of documentation (29) Discussion of the solution: strategy (29) Long, complex change (25) Lack of context (19) Discussion of the solution: correctness (14) Impact of change (11) Irreproducible bug (6) Lack of tests (5)	<u>Communication (18)</u> Communicative intention (9) Language issues (3) Propagation of confusion (3) Fatigue (1) Noisy work environment (1)	Lack of familiarity with existing code (47) Lack of programming skills (40) Lack of understanding of the problem (21) Lack of understanding of the change (17) Lack of familiarity with the technology (14) Lack of knowledge about the process (3)
Reasons 30 topics (507)				
	Delaying (31) Decreased review quality (11) Additional discussions (11) Blind approval (8) Review rejection (4) Increased development effort (4) Assignment to other reviewers (2)	Better solution (1) Incorrect solution (1)	Decreased confidence (10) Abandonment (6) Frustration (5) Anger (2) Propagation of confusion (2)	
Impacts 14 topics (86)				
	Improved organization of work (5) Small, clear changes (4) Improved documentation (4) Assignment to other reviewers (1) Blind approval (1)	Improved documentation (4)	Information requests (36) Off-line discussions (12) Follow-up questions/suggestions (10) Disagreement resolution (6)	Improved familiarity with existing code (28) Testing the change (5) Improved familiarity with the technology (2)
Coping strategies 15 topics (116)				

没有特别有效的策略

评审者需要哪些信息？ [12]



代码重构的审查补丁如何？

- ◆ 施乐公司的工业案例研究[23]
- ◆ "应对糟糕的设计和编码风格是开发者在代码变更中进行重构的主要驱动力。然而，功能变更和修复缺陷的活动也常常促使开发者重构代码。"

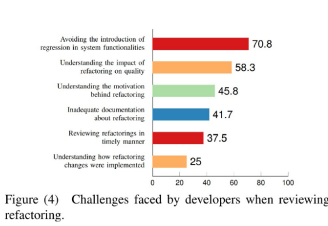


Figure (4) Challenges faced by developers when reviewing refactoring.

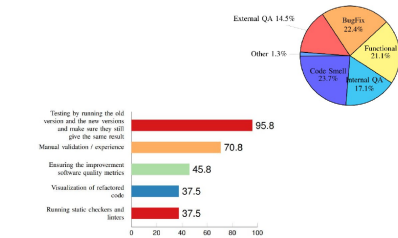


Figure (5) Mechanisms used to ensure the correctness after the application of refactoring.

知识转移呢？

- ◆ 增加代码审查轮次能否提升开发者的代码质量？
 - 遗憾的是，目前无论是单个项目内部还是跨项目层面，均未发现显著证据。 [16]
- ◆ 那么，让我们看看大型科技公司的代码审查机制。请参阅后续几张幻灯片。

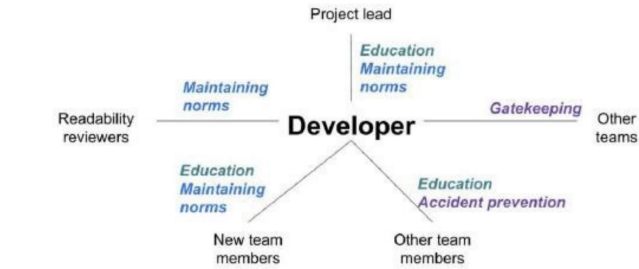
代码审查最佳实践[1, 3, 7]

- ◆ 当代同行评审遵循轻量化、灵活的流程
- ◆ 代码审查应在早期（提交变更前）、及时且频繁地进行
- ◆ 审阅者事先了解上下文和代码，能更快完成审阅并为作者提供更有价值的反馈
- ◆ 变更规模较小
- ◆ 审阅者数量较少（1至5人）
- ◆ 阅读代码及补丁的非正式讨论与评论
- ◆ 审查不仅限于发现组级缺陷：还包括问题解决（方案开发）、代码可读性与可维护性、遵循规范（所属公司标准）、事故预防、把关（召回率）

代码审查中主持人的角色？

29

MCR最佳实践[3]



- ◆ 上图展示了谷歌代码项目中开发者（即作者）与审阅者之间的关系

30

谷歌MCR最佳实践[3]

- ◆ 建议评审人：请能够担任评审的人员轮流担任评审，采用循环轮换的方式。
 - 避免长期过度占用团队中最佳人选的精力
- ◆ 通过代码分析补充人工评审
 - 应用风格检查器、代码检查工具
 - 例如：Java的Checkstyle
- ◆ 评审频率：每周4小时评审（及4次评审）（中位数）
- ◆ 代码审查速度：作者等待1-5小时
- ◆ 变更代码规模：1行代码（约10%）、单个文件（约30%）、

30

少于10个文件（约90%）。

MCR最佳实践

候选审阅者？

- 代码所有者
- 曾对代码进行过修改的开发者
- 先前参与代码开发的开发者（例如通过结对编程产出代码者）
- 经验丰富的审阅者
- 在其他代码中开发过类似变更片段功能的开发者
- ...

重大变更请求（MCR）问题

- 代码质量与代码规模
 - 复杂度较高的代码在代码审查中获得的讨论和反馈较少。
 - 代码评审反馈越少，发布后出现的缺陷就越多
- 距离
 - 导致代码审查流程延迟或引发误解
 - 方面一：地理因素
 - 作者与审阅者之间存在较大物理距离
 - 方面二：组织问题
 - 审稿人来自不同团队或承担不同职责
- 社交互动
 - 语气：负面语气评论的实用性较低
 - 权力：要求他人修改评论。作者感到不满

紧急修复 vs. 全面解决方案 vs. 高级草图)

背景

- 因代码变更预期不匹配导致的误解（例如：可有可无的功能）

无效代码变更请求的其他问题：MCR反模式[18]

TABLE I: MCR anti-patterns and their associated symptoms and consequences.

MCR anti-pattern	Symptoms	Potential Consequences
Confused reviews (CR)	- Reviewers ask question(s) about the rationale or the solution approach of the patch. - Reviewers express incertitude in review comments.	- Process: Confusion decreases the efficiency and the effectiveness of the review [6], [10]. - Artifact: The patch may still have poor quality after the review as reviewers may not fully understand the patch [6]. - People: Reviewers may feel frustrated and may express negative sentiments due to the confusion [6].
Divergent reviews (DR)	- The review decisions do not reach consensus. - The review scores are diverged. - Reviewers post conflict review comments to each others.	- Process: The divergence can slow down the integration process [16]. - Artifact: It is correlated with negative development outcomes (i.e., patches without changing) [16]. - People: The divergence increases contributors' likelihood of leaving the communities and lead to the poor team performance [11], [29].
Low review participation (LRP)	- The patch does not have other developers as a reviewer. - The patch receive few/short comments. - The patch does not receive promptly feedback from reviewers.	- Process: The lack of review participation has a negative impact on review efficiency and effectiveness [30]. - Artifact: Patches with low number of reviewers can be more defect-prone [7]. - People: The inefficient reviewer feedback could make the patch author forget the change. [5]

TABLE II: The detection results of MCR anti-patterns.

Anti-pattern	# Instances	Kappa	Agreement
Confused Reviews (CR)	21	0.93	Perfect
Divergent Reviews (DR)	20	1.0	Perfect
Low Review Participation (LRP)	32	1.0	Perfect

TABLE III: The prevalence of MCR anti-patterns.

Category	Count
Code reviews affected by one anti-pattern	67
Code reviews affected by two anti-patterns	21
Code reviews affected by three anti-patterns	4

是否也应审查测试代码？

◆ 简单回答是"应该"，但需调整关注重点且频率较低（需与生产代码同步审查）

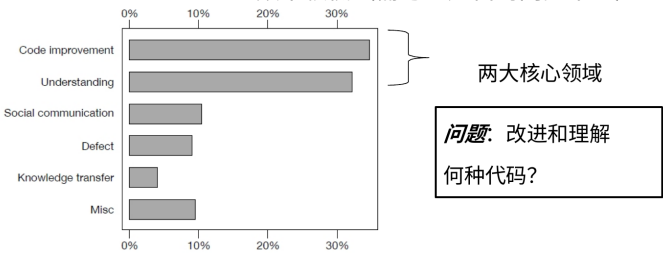


Figure 1: The outcomes of comments in code review of test files, after manual analysis in 600 comments (CL=99%, CI=5).

通过审查测试代码来理解和改进哪些类型的代码？ [15]

由德国研究人员对92名开发者开展。

INTERVIEWEES' EXPERIENCE (IN YEARS) AND WORKING CONTEXT				
ID	Developer	Reviewer	Working context	Applying TDR
P1	8	8	OSS	Almost never
P2	3	3	Company A	Almost Always
P3	15	15	Company A	Almost Always
P4	10	10	Company B	Almost Never
P5	10	5	Company C	Always
P6	3	2	Company D	Sometimes
P7	16	16	Company E	Always
P8	4	4	Company F / OSS	Sometimes
P9	3	3	Company G / OSS	Never

Finding 1. In reviews in which tests are presented first, participants found significantly more bugs in test code, yet fewer maintainability issues in production code. The production bugs and test maintainability issues found is stable across the treatments.

Finding 3. Perceived problems with TDR: Developers report to (1) consider tests as less important than production, (2) not being able to extract enough knowledge from tests, (3) not being able to start a review from tests of poor quality, and (4) being comfortably used to read the patch as presented by their code review tool.

Finding 4. Perceived advantages of TDR: Developers report that TDR (1) allows them to have a concise, high-level overview of the code under test and (2) helps them in being more testing-oriented, hence improving the overall test code quality.

现代代码审查：

课堂活动 MCR-2

<https://github.com/marketplace/category/code-review>

- ◆ 组成4-5人小组
 - 代码作者A：1名学生
 - 代码审阅者R：无预设角色
 - 成员间协商确定评审工具（我们使用Github支持）
- ◆ 时间线（单位：分钟）
 - 00-15：A 在 Github 平台为代码 C 准备补丁 P，附上补丁说明，并通过电子渠道通知组员
 - 15-25分钟：R审阅P并在Github发布评论。审阅完成后通过电子渠道通知
 - 15-25：A 逐条回应 R 提出的疑问并修订补丁 P' 进行迭代修订
 - 25-30：R 批准 P，A 应用最终补丁 P' 获得最终修正代码 C2
 - 30：A将修正后的C2附带评论（图片或Github仓库链接？）提交至Canvas讨论区供同行共享，同时将C2作为提交

自学家庭练习 [无需提交]

◆ 阅读参考文献[1]

- 探索提升团队协作意识的方法
- 第5页资深开发者总结的代码审查五大益处是什么？
- 第5-6页中，团队可采取哪些措施促进知识转移？
- 第7页中，如果代码审查质量较低，会出现哪些问题？
- 第9页提出了哪些建议？你是否认同？

◆ 阅读参考文献[5]

- 第23页，幻灯片演示作者认为该代码审查支持工具备哪些核心功能？幻灯片演示作者认为该工具具备？

◆ 自我反思

- 若公司要求你在项目中实施MCR但未提供额外预算（\$），你将如何实施MCR？
- 距离是MCR团队面临的主要问题。当团队成员分布在不同城市且可能拥有不同文化背景）如何开展MCR？

37

课后阅读

[1] Alberto Bacchelli 与 Christian Bird. 2013. 现代代码审查的期望、成果与挑战. 载于《2013年国际软件工程会议论文集》(ICSE '13). IEEE出版社, 美国新泽西州皮斯卡塔韦, 712-721页. <https://ieeexplore.ieee.org/document/6606617/>

38

参考文献与资源

1. 阿尔贝托·巴切利与克里斯蒂安·伯德。2013。《现代代码审查的期望、成果与挑战》。载于《2013年国际软件工程会议论文集（ICSE '13）》。IEEE出版社, 美国新泽西州皮斯卡塔韦, 712-721页。
2. 克里斯托弗·汤普森和大卫·瓦格纳。2017。《开源项目中现代代码审查与安全的大规模研究》。载于第13届软件工程预测模型与数据分析国际会议（PROMISE）论文集。ACM, 美国纽约州纽约市, 83-92页。DOI: <https://doi.org/10.1145/3127005.3127014>
 - 代码审查似乎并非识别安全漏洞的有效手段
3. Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, Alberto Bacchelli. 2018. 现代代码审查：谷歌案例研究. 收录于第40届国际软件工程实践会议（ICSE-SEIP '18）论文集 ACM, 美国纽约州纽约市, 181-190.DOI: <https://doi.org/10.1145/3183519.3183525>
4. Davide Spadini, Mauricio Aniche, Margaret-Anne Storey, Magiel Bruntink, and Alberto Bacchelli. 2018. 当测试遇上代码审查：开发者为何及如何审查测试. 收录于《第40届国际软件工程会议论文集（ICSE '18）》。ACM出版社, 美国纽约州纽约市, 677-687页。DOI: <https://doi.org/10.1145/3180155.3180192>
5. SlideShare: <https://www.slideshare.net/excellaco/modern-code-review>

39

参考文献与资源

6. M. E. Fagan. 1976. 设计与代码审查以减少程序开发中的错误. IBM系统期刊, 15卷, 3期（1976年9月）, 182-211页。DOI=<http://dx.doi.org/10.1147/sj.153.0182>
7. 彼得·C·里格比与克里斯蒂安·伯德。2013。《当代软件同行评审实践的趋同现象》。载于《2013年第九届软件工程基础联合会议论文集（ESEC/FSE 2013）》。ACM, 美国纽约州纽约市, 202-212页。DOI: <https://doi.org/10.1145/2491411.2491444>
8. 维基百科。代码审查工具列表。 https://en.wikipedia.org/wiki/List_of_tools_for_code_review. 2018年。
9. Gerrit: <https://www.gerritcodereview.com/> 最后访问时间 2021
10. Review Board: <https://www.reviewboard.org/> 最后访问时间 2021
11. Phabricator <https://www.phacility.com/> 最后访问时间 2021
12. 卢卡·帕斯卡雷拉、达维德·斯帕迪尼、法比奥·帕隆巴、马吉尔·布伦廷克与阿尔贝托·巴切利。2018。《当代代码审查中的信息需求》。ACM人机交互会议论文集第2卷, CSCW专题, 第135篇（2018年11月）, 27页。DOI: <https://doi.org/10.1145/3274404>
13. 阿丘德·拉姆、阿南德·阿肖克·萨万特、马尔科·卡斯特卢乔与阿尔贝托·巴切利。2018。《何种因素使代码变更更易审查：代码变更可审查性的实证研究》。载于《2018年ACM欧洲软件工程会议暨软件工程基础研讨会联合会议论文集（ESEC/FSE 2018）》。201-212页。DOI: <https://doi.org/10.1145/3236024.3236080>

进化与再造, SANER 2019年会议论文集, 第49-60页。 <https://doi.org/10.1109/SANER.2019.8668024>

40

14. Felipe Ebert, Fernando Castor, Nicole Novielli, Alexander Serebrenik: 代码审查中的困惑：成因、影响及应对策略。收录于第26届IEEE国际软件分析会议论文集，

参考文献与资源

15. Davide Spadini, Fabio Palomba, Tobias Baum, Stefan Hanenberg, Magiel Bruntink, and Alberto Bacchelli. 2019. 测试驱动代码审查：一项实证研究. 收录于第41届国际软件工程会议（ICSE '19）论文集. IEEE出版社, 1061–1072.DOI:<https://doi.org/10.1109/ICSE.2019.00110>

16. 玛丽亚·卡乌洛、林斌、加布里埃莱·巴沃塔、朱塞佩·斯卡尼耶洛、米凯莱·兰扎。现代代码审查中的知识转移。载于第28届IEEE/ACM国际程序理解会议（ICPC 2020）论文集。<https://binlin.info/downloads/Caulo2020a.pdf>

17. Github 文档：<https://docs.github.com/en/github> 最后访问时间 2021

18. Moataz Chouchen, Ali Ouni, Raula Gaikovina Kula, Dong Wang, Patanamon Thongtanunam, Mohamed Wiem Mkaouer, Kenichi Matsumoto. 2021. 《现代代码审查中的反模式：症状与流行趋势》. SANDER 2021, pp 531-535, 2021.

19. 妮可·达维拉，英格丽德·努涅斯。现代代码审查的系统文献综述与分类学。《系统与软件期刊》177 (2021): 文章编号 110951。<https://doi.org/10.1016/j.jss.2021.110951>

20. 李瑞茵、穆罕默德·索利曼、梁鹏、帕里斯·阿夫杰里乌。《代码审查中架构侵蚀的症状：对两个OpenStack项目的研究》。2022年IEEE第19届国际软件架构会议（ICSA 2022），第24-35页，2022年。

21. 格伦福德·J·迈尔斯。程序测试与代码走查检查的控制实验。《计算机协会通讯》，1978 21(9): 760-768, 1979.

22. Venkatesh Potluri, Maulishree Pandey, Andrew Begel, Michael Barnett, Scott Reitherman: 代码漫步：促进混合能力协作软件开发中的共享认知。ASSETS 2022:²⁹11-20:16

进化与再造，SANER 2019年会议论文集，第49-60页。<https://doi.org/10.1109/SANER.2019.8668024>

参考文献与资源

23. 埃曼·阿卜杜拉·奥马尔、侯赛因·阿尔鲁拜、穆罕默德·维姆·姆卡乌尔、阿里·乌尼、马鲁安·凯森蒂尼：现代代码审查背景下的重构实践：施乐公司的工业案例研究。ICSE (SEIP) 2021: 348-357